CONTRAT_GEN

Documentation technique

Architecture, fonctionnement interne et maintenance

Pour l'installateur et le développeur

Édition du 13 octobre 2026

Sommaire

1. Architecture générale
2. Principes de conception
3. Structure du projet
4. Machine à états et journal
5. Stockage : le disque est la base
6. Configuration d'une instance
7. Moteur de contrats
8. Mails
9. Sécurité
10. Routes
11. Installation et déploiement
12. Exploitation
13. Tests
14. Étendre une instance
15. Dossiers importés : SharePoint et Excel

1. Architecture générale

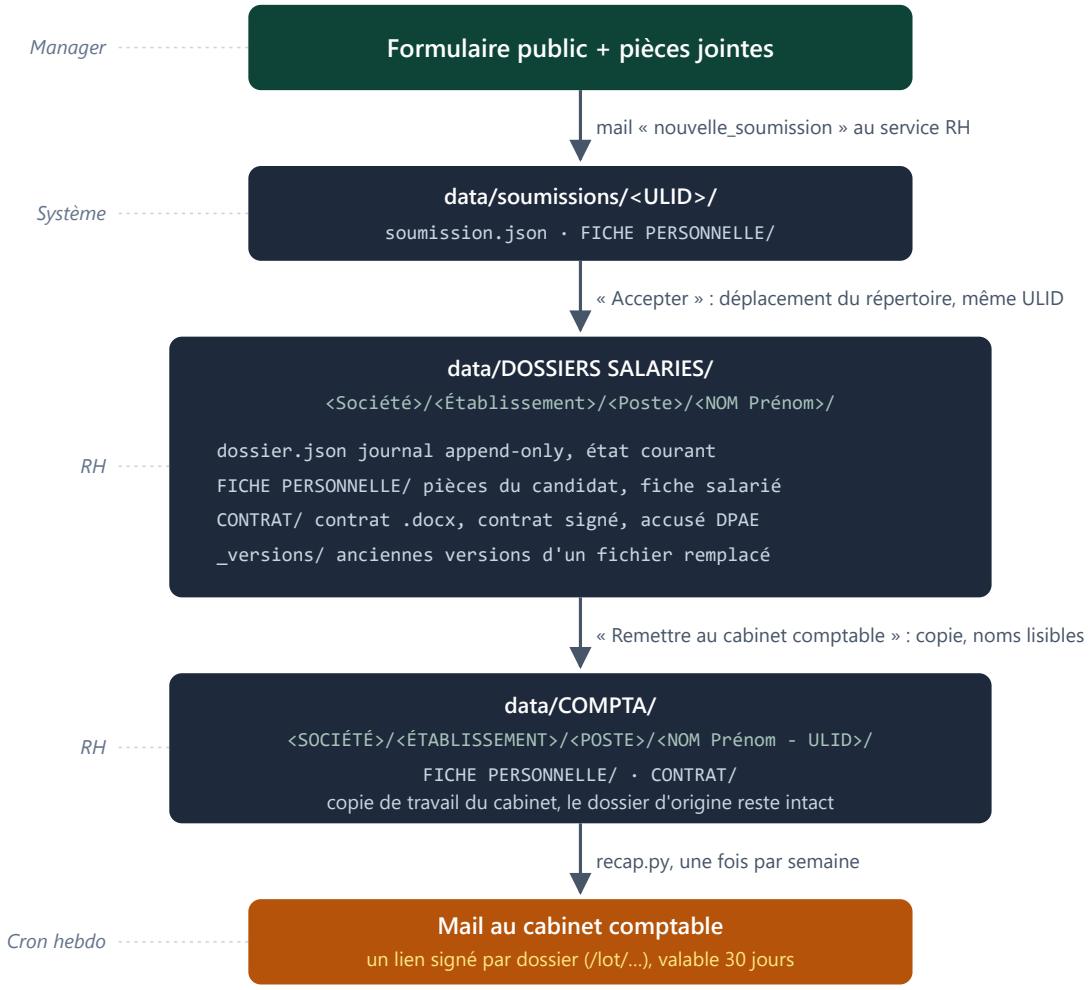
Principe fondamental

Un client = une copie du dépôt. Le code est le produit ; l'instance (configuration + données) vit à part et n'est jamais versionnée. Aucune donnée n'est mutualisée entre clients.

Stack

Couche	Technologie
Runtime	Python 3.12
Framework web	Flask, templates Jinja2
Serveur WSGI	Waitress en production (mono-process, 4 threads) ; Werkzeug en développement (`DEBUG=1`)
Contrats	python-docx (modèles `.docx`), htmldocx (modèles `.html`), num2words (montants en lettres)
Signature électronique	Yousign, optionnel, désactivé par défaut
Stockage	Fichiers JSON et arborescence disque. **Aucune base de données.**
Dépendances	`requirements.txt`, versions épinglées

Flux



Aucune base de données : le disque est la base, dossier.json fait foi sur l'état.

Du formulaire public au cabinet comptable

---

2. Principes de conception

2.1 Le disque est la source de vérité

`dossier.json` fait foi sur l'état : c'est la dernière entrée `vers` du journal. Un fichier présent sur disque (contrat, accusé) est un indice, jamais une preuve d'état. Le journal ne fait que croître.

2.2 Mono-process obligatoire

`store._verrou(uid)` est un verrou de thread : il n'exclut qu'à l'intérieur d'un process. Deux serveurs sur le même stockage perdraient des entrées de journal en silence. Waitress est mono-process par construction ;

ne jamais lancer `gunicorn` avec plusieurs workers ni `waitress-serve --processes`. Les plafonds de rate-limit vivent aussi en mémoire du process.

2.3 Configuration pure données

Tout ce qui varie d'un client à l'autre vit dans `config/` : JSON, modèles de contrats, modèles de mails. Aucun `.py` n'y entre. L'application ne réécrit jamais sa configuration ; seul `configurer.py compte` ajoute un compte dans `instance.json`.

2.4 Garde-fou au démarrage

`config.verifier()` refuse de servir tant que la configuration est incomplète ou dangereuse :

- version de configuration incompatible ;
- secret HMAC encore à sa valeur d'exemple ;
- compte RH au mot de passe d'exemple, aucun compte, ou bloc `utilisateurs` malformé ;
- aucun établissement déclaré ;
- `mails.rh` ou `mails.expediteur` vide, `url` publique vide (les liens de tous les mails partiraient vides) ;
- rôle du formulaire inconnu ou pointant un champ inexistant, règle `derives` malformée ;
- jeton d'un modèle de contrat sans source ;
- poste sans modèle de contrat, grille de salaire incomplète ;
- bloc `stockage` ou `conservation` malformé, dossier de stockage inexistant ou non inscriptible ;
- `theme.json` malformé : contraste insuffisant (calculé sur les couples texte/fond réels), valeur hors bornes, clé inconnue, ou fichier de `marque/` déclaré mais absent.

---

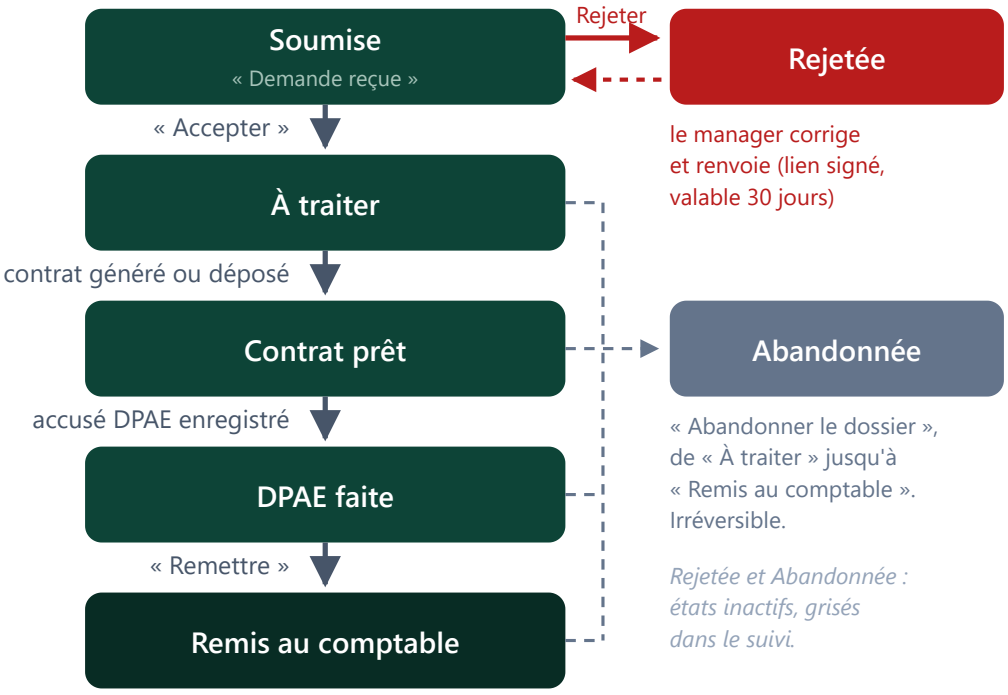
3. Structure du projet

```
Contrat_Gen/
├── app.py                    # Routes Flask, parcours, rate-limit, verrou multi-machine
├── store.py                  # Stockage disque, ULID, journal, transitions, liens signés, purge
├── contrat.py                # Moteur de remplissage des modèles (.docx / .html -> .docx)
├── config.py                 # Lecture de config/, cache, garde-fou verifier()
├── mails.py                  # Rendu des modèles .txt, envoi SMTP ou console
├── signature.py              # Signature électronique Yousign (optionnel)
├── installer.py              # Crée l'instance d'un nouveau client
├── configurer.py             # Bilan de la config, assistant, comptes, test SMTP
├── placeholders.py           # Fiche des {{Jetons}} valides, dérivée de la config
├── doctor.py                 # Génère un contrat d'essai par croisement de la config
├── recap.py                  # Mail hebdomadaire au cabinet comptable (cron)
├── sharepoint.py             # Import des salariés déjà en poste depuis un SharePoint (optionnel)
├── excel.py                  # Lecture des exports Excel RH pour compléter les dossiers importés
├── requirements-sharepoint.txt  # msal + requests, seulement avec un bloc `sharepoint`
├── purger.py                 # Aperçu de la purge en ligne de commande (lecture seule)
├── demo.py                   # Démo sur config.demo/ et data_demo/
├── templates/                # Pages Jinja2 (formulaire, suivi, dossier, lot, recuperer, completer…)
├── config/                   # Instance de démonstration versionnée (fixture des tests)
├── config.exemple/           # Squelette copié par installer.py
├── config.demo/              # Configuration riche pour demo.py (modèles HTML, grille)
├── data/                     # Données de production (jamais versionné)
├── data_demo/                # Données de la démo, effacées à chaque lancement
├── docs/                     # Guides Markdown et PDF (docs/pdf/build.py)
└── tests/                    # Suite de tests, sans framework : python tests/test_x.py
```

Responsabilités et dépendances internes

Module	Responsabilité	Importe
`app.py`	Routes, contrôle d'accès, rate-limit, effets de bord des transitions (mails, fiche, contrat)	config, store, contrat, mails, signature
`store.py`	Répertoires, journal, transitions permises, noms de fichiers lisibles, copie comptable, liens HMAC, purge	config
`contrat.py`	Valeurs des jetons, règles dérivées, formats, génération `.docx`	config
`config.py`	Chargement JSON, rôles du formulaire, mentions, grille, garde-fou	contrat, store (imports locaux : le cycle est assumé)
`mails.py`	Modèles `.txt`, transport SMTP (STARTTLS vérifié) ou console	config, store
`sharepoint.py`	Connexion Microsoft (device flow), lecture Graph, import en tâche de fond	config, store (`msal`, `requests` à la demande)
`excel.py`	Lecture `.xlsx` (stdlib), conversion des colonnes vers les champs du formulaire	config, sharepoint (noms)

4. Machine à états et journal



États et transitions, tels que définis dans store.py

États

```
ETATS = ["Soumise", "Rejetee", "ATraiter", "ContratPret",
         "DpaeFaite", "RemisComptable", "Abandonnee", "Parti"]
INACTIFS = {"Rejetee", "Abandonnee", "RemisComptable"}   # grisés dans le suivi
```

Transitions permises

`store.transition()` est le point de passage unique ; une transition absente de la table lève `ValueError`, même sur un POST forgé.

```
TRANSITIONS = {
    "Soumise":        {"Abandonnee"},
    "Rejetee":        {"Abandonnee"},
    "ATraiter":       {"ContratPret", "Abandonnee"},
    "ContratPret":    {"DpaeFaite", "Abandonnee"},
    # etat herite : dossiers ecrits avant la suppression de l'etape, jamais atteint
    "ContratSigne":   {"DpaeFaite", "Abandonnee"},
    "DpaeFaite":      {"RemisComptable", "Abandonnee"},
    "RemisComptable": {"Abandonnee"},
}
```

Trois passages ne transitent pas par cette table car ils changent de répertoire ou de zone : `store.valider()` (Soumise → ATraiter, la soumission devient un dossier salarié), `store.rejeter()` (→ Rejetee) et `store.resoumettre()` (Rejetee → Soumise, correction). L'interface n'offre l'abandon qu'à partir de « À traiter ».

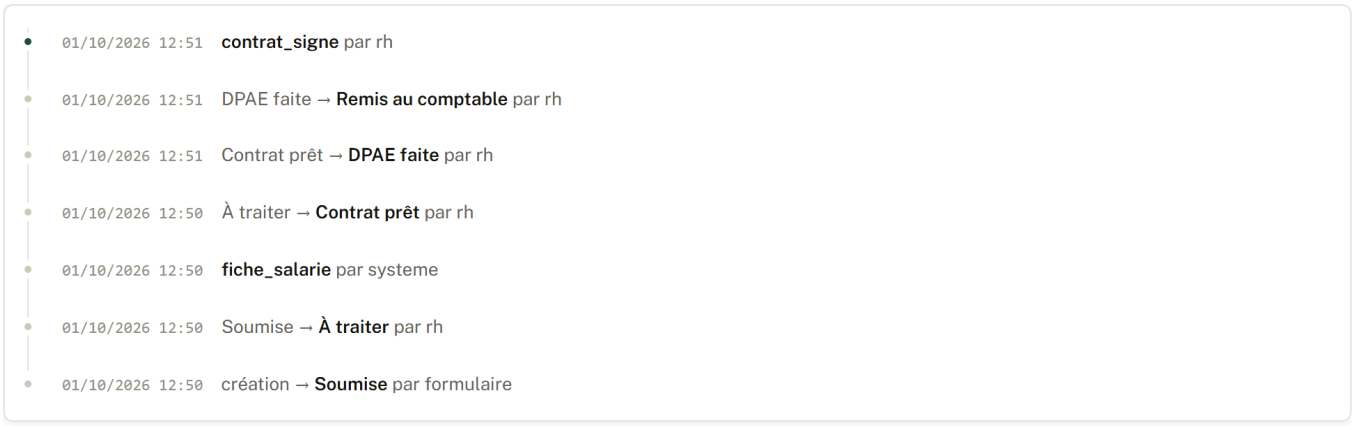
Le journal

Chaque entrée porte au minimum `de`, `vers`, `le` (ISO 8601 UTC) et `par`. Une entrée de transition change `vers` ; une entrée d'événement garde `de == vers` et porte un `type`. Extrait réel d'un dossier de la démo :


```
{
  "formulaire_version": 1,
  "id": "01M2N3D8WJAFCSD5EG1E15XVNY",
  "champs": {"etablissement": "Basilic Café / Paris Bastille", "poste": "Equipier Polyvalent",
             "nom_prenom": "BENALI Sarah", "date_debut": "2026-10-12", "...": "..."},
  "link_epoch": 0,
  "lien_comptable_epoch": 0,
  "journal": [
    {"de": null, "vers": "Soumise", "le": "2026-09-16T12:35:02+00:00", "par": "formulaire"},
    {"de": "Soumise", "vers": "ATraiter", "le": "2026-09-16T12:35:20+00:00", "par": "rh"},
    {"de": "ATraiter", "vers": "ContratPret", "le": "2026-09-16T12:35:20+00:00", "par": "rh",
     "modele": "Equipier.html"},
    {"de": "ContratPret", "vers": "DpaeFaite", "le": "2026-09-16T12:35:24+00:00", "par": "rh"},
    {"de": "DpaeFaite", "vers": "RemisComptable", "le": "2026-09-16T12:35:25+00:00", "par": "rh"},
    {"de": "RemisComptable", "vers": "RemisComptable", "le": "2026-09-16T12:35:27+00:00", "par": "rh",
     "type": "contrat_signe"},
    {"de": "RemisComptable", "vers": "RemisComptable", "le": "2026-09-16T12:35:29+00:00",
     "type": "acces_lot", "par": null, "ip": "127.0.0.1"}
  ]
}
```


Types d'événements écrits par `store.noter()` : `fiche_salarie`, `fiche_echouee`, `mail_echoue`, `signature_envoyee`, `contrat_signe` (contrat signé déposé, non bloquant, seulement en `RemisComptable`, sans changer d'état ; exclu de la copie `COMPTA/` et du lot par `store.fichiers_compta()`), `renvoi_comptable`, `correction_renvoyee` (lien de correction renvoyé après un mail de rejet raté), `acces_lot`. S'y ajoute `purge`, posé par `store.purger()` au moment où il allège le journal. Les dossiers importés ajoutent `import` (première entrée du journal), `date_entree`, `correction` et `completion_excel` (voir section 15).

Une remise au comptable est reconnue par `store.est_remise()` : `vers == RemisComptable` **et** `de != RemisComptable`, ou `renvoi_comptable`. `noter()` écrit aussi `vers: RemisComptable` sur un dossier déjà remis (accès au lot, contrat signé…) : ces notes ne sont pas des remises, sans quoi l'ouverture du lot par le cabinet le renverrait au récapitulatif suivant. Une remise fige en plus son **jour local** dans `jour` (`le` est en UTC : entre 0 h et 2 h à Paris, il nommerait les fichiers du lot de la veille) ; `store.date_remise()` lit `jour` avant `le`. Un rejet ajoute `motif` et `commentaire` ; une correction ajoute `motif: "correction"`.



Le journal tel qu'affiché dans la fiche du dossier

---

5. Stockage : le disque est la base

Arborescence réelle

```
data/
├── .serveur-actif.json                # verrou multi-machine
├── recap.json                         # mail hebdo : dernier envoi réussi / échec par cabinet (écrit
par recap.py seul)
├── soumissions/<ULID>/                # demandes pas encore acceptées
│   ├── soumission.json
│   └── FICHE PERSONNELLE/             # pièces au nom de leur rôle (identite.pdf, rib.pdf…)
├── DOSSIERS SALARIES/
│   └── <Société>/<Établissement>/<Poste>/<NOM Prénom>/
│       ├── dossier.json               # journal append-only
│       ├── FICHE PERSONNELLE/         # pièces renommées, fiche salarié
│       ├── CONTRAT/                   # contrat .docx, accusé DPAE (jamais le contrat signé)
│       └── _versions/                 # fichiers remplacés, horodatés
├── COMPTA/
│   └── <SOCIÉTÉ>/<ÉTABLISSEMENT>/<POSTE>/<NOM Prénom - ULID>/
│       ├── FICHE PERSONNELLE/
│       └── CONTRAT/
└── mails/                             # mode console : un .eml par envoi
```

Le premier niveau est le `groupe` de l'établissement (à défaut, le nom de la société). L'acceptation **déplace** le répertoire de `soumissions/` vers `DOSSIERS SALARIES/` en gardant l'ULID ; `soumission.json` devient `dossier.json`. Les clés internes des buckets (`pieces` , `contrat` , `_versions`) sont traduites en noms de répertoires lisibles par `store.BUCKETS` . Un fichier déposé par-dessus un existant est archivé dans `_versions/` avant remplacement.

Le mode `stockage.mode = "dossier"` place `data/` dans un répertoire répliqué par un client de synchronisation (OneDrive, SharePoint) ; le garde-fou vérifie alors que le répertoire existe et accepte l'écriture avant de servir. La variable d'environnement `DONNEES` , si elle est posée sur la ligne de lancement, l'emporte sur ce bloc (`config._donnees()`) : une instance lancée avec `DONNEES=` écrit là, pas sur le drive.

Identifiants

`store.nouvel_id()` produit un ULID : 48 bits de temps en millisecondes puis 80 bits d'aléa, encodés sur 26 caractères en base 32 Crockford (sans I, L, O, U). Triable par date de création, unique sans coordination.

Verrou multi-machine

`.serveur-actif.json` empêche deux serveurs de servir le même stockage :

```
{"hote": "poste-rh", "pid": 12345, "le": 1789562000.0}
```

Sur la même machine, le PID est sondé (vivant ou mort). Depuis une autre machine, le verrou est considéré périmé après 5 minutes sans rafraîchissement. Un démarrage refusé affiche l'hôte qui tient le verrou.

---

6. Configuration d'une instance

Fichiers

Fichier	Contenu
`instance.json`	Identité du client, secret HMAC, URL publique, mails, comptes, motifs de rejet, modèles par poste, règles dérivées, saisie RH, stockage, conservation, `sharepoint` (optionnel)
`formulaire.json`	Champs du formulaire public et rôles
`societes.json`	Sociétés, établissements, mentions légales, couloir de contrat, adresses du manager et du cabinet. Avec un bloc `supabase`, c'est la **copie de secours** de la base (voir Sociétés et établissements dans Supabase)
`theme.json`	**Optionnel** — direction artistique du client : palette, logo, favicon. Absent = la DA d'origine
`marque/`	**Optionnel** — logo et favicon du client, servis par Flask sur `/marque/` (`static_folder` de l'instance)
`grille.json`	Grille de rémunération (optionnel)
`contrats/`	Modèles de contrat `.docx` ou `.html`
`mails/`	Modèles de mails `.txt`
`contrats/fiche_salarie.docx`	Modèle de la fiche salarié du CLIENT (optionnel ; le fichier vit dans `contrats/`, la clé `fiche_salarie` d' `instance.json` le nomme). Sans elle, le modèle générique versé avec le code (`modeles/fiche_salarie.docx`) sert à toutes les instances — la fiche est **toujours** produite à la validation, avec l'un ou l'autre modèle
`PLACEHOLDERS.md`	Fiche des jetons valides, générée par `placeholders.py`, à remettre au client

`instance.json`

```
{
  "config_version": 1,
  "client": "Basilic Café",
  "secret": "<64 caractères hexadécimaux, générés par installer.py>",
  "url": "https://embauche.basiliccafe.fr",
  "signature": {"mode": "manuel"},
  "stockage": {"mode": "local"},
  "conservation": {"jours": 1095, "jours_candidature": 730,
                   "apres": ["RemisComptable", "Rejetee", "Abandonnee"]},
  "fiche_salarie": "fiche_salarie.docx",
  "mails": {
    "mode": "smtp", "hote": "smtp.basiliccafe.fr", "port": 587,
    "utilisateur": "rh@basiliccafe.fr", "mot_de_passe": "…", "expediteur": "rh@basiliccafe.fr",
    "rh": ["rh@basiliccafe.fr"], "dpae": null, "recap": null,
    "comptable_defaut": "compta@cabinet.fr"
  },
  "utilisateurs": {"rh": {"mdp_hash": "scrypt:32768:8:1$…"}},
  "motifs_ko": ["Pièce illisible ou manquante", "Informations incohérentes",
                "Établissement ou poste erroné", "Rémunération à revoir", "Autre"],
  "templates": [
    {"quand": {"poste": "Equipier Polyvalent", "temps_partiel": "OUI"},
     "modele": "Equipier_Partiel.html"},
    {"quand": {"poste": "Equipier Polyvalent"}, "modele": "Equipier.html"},
    {"quand": {"poste": "Manager"}, "modele": "Manager.html"}
  ],
  "critiques": ["SalaireChiffres", "SalaireLettres"],
  "saisie_rh": [{"placeholder": "NumeroCarteBTP", "libelle": "N° carte BTP",
                 "aide": "au dos de la carte"}],
  "derives": [
    {"placeholder": "Nationalite", "si": ["nationalite", "==", "Autres"],
     "alors": "{{NationaliteEtrangere}}", "sinon": "française"},
    {"placeholder": "DureeMensuelle", "format": "mensualise", "de": "TempsTravail"}
  ]
}
```

Notes :

- `templates` accepte aussi la forme simple `{"Manager": "Manager.docx"}`. En forme liste, la **première** règle dont tous les `quand` correspondent aux champs du dossier l'emporte : mettre les cas particuliers avant le cas général. Une règle `{"quand": {...}, "modele": null}` marque une **exclusion volontaire** (croisement identifié, mais sans contrat à produire) plutôt que d'omettre la règle ; `doctor` l'affiche « exclu (volontaire) », distinct d'un cas oublié (`✗`). Les deux formes lisent `null` pareil : en forme simple, `{"Manager": null}` exclut le poste de la même façon.
- Le garde-fou de démarrage (`_verifier_regles`) refuse une règle `templates` en liste dont une clé de `quand` n'est pas un champ du formulaire, dont la valeur ne correspond à aucune option du champ visé, ou — pour le champ `etablissement` — qui écrit le nom d'un site seul au lieu de la clé complète « Société / Établissement » (il propose alors la bonne clé). Une règle ou un `quand` qui n'est pas un objet est refusé sans faire planter le garde-fou lui-même.
- `stockage.mode` vaut `local` (répertoire `data/` de l'instance) ou `dossier` avec une clé `chemin` (répertoire synchronisé).

- Le contrat signé ne bloque rien (à recueillir tout de même) : la DPAE se déclare dès `ContratPret`. `store.signe_manquant()` décide à la fois du badge « signé manquant » (« Salariés ») et de l'autorisation du dépôt manuel ou de l'envoi Yousign : état dans `store.SIGNE_DEPOSABLE` (`RemisComptable` seulement : dépôt depuis la fiche salarié), dossier non purgé, aucun signé déjà déposé. L'ancienne clé `signature.avant_dpae` est ignorée ; `doctor` la signale comme obsolète sans changer son verdict. Les dossiers déjà écrits à `ContratSigne` s'affichent sur l'étape « Contrat prêt » et passent à la DPAE.
- `conservation` absent = aucune purge. `jours` et `jours_candidature` sont des entiers strictement positifs ; `apres` liste des états de `store.ETATS`.
- `sharepoint` : facultatif ; absent = pas de boutons « Récupérer les dossiers » ni « Compléter les dossiers » dans l'onglet Établissements. Clés détaillées en section 15.
- `critiques` : jetons qui doivent avoir une valeur non vide à la génération, en plus des champs requis du formulaire.
- `saisie_rh` : jetons qu'aucun champ ne fournit ; la RH les saisit sur l'écran « À traiter » avant de générer.
- `fiche_salarie` : facultatif. Absent, le modèle générique `modeles/fiche_salarie.docx` (versé avec le code, hors `config/`) sert par défaut : état civil, titre de séjour, coordonnées, poste et contrat, disponibilités, pièces. À la génération, une ligne **de tableau** dont aucun mot-clé n'a de valeur (absent de la config, ou réponse vide) est retirée, et une section vide disparaît avec son titre ; `doctor` liste les lignes que la config n'alimente pas. L'élagage ne concerne que les tableaux : un mot-clé placé dans la prose, un titre ou un en-tête reste obligatoire et fait refuser la fiche s'il n'est pas alimenté. Deux lignes acceptent deux noms de mot-clé : `{{NumSS}}` / `{{NumeroSecu}}` et `{{Domicile}}` / `{{Adresse}}`. Ses **mots-clés** ne sont donc pas vérifiés au démarrage — l'élagage les rend facultatifs — mais sa **présence** l'est : `modeles/fiche_salarie.docx` manquant refuse le démarrage, sinon la fiche échouerait à chaque validation. Un modèle déclaré par le client est vérifié mot-clé par mot-clé, comme un contrat.

`formulaire.json`

```
{
  "version": 1,
  "titre": "Formulaire d'embauche",
  "intro": "À remplir par le manager…",
  "champs": [
    {"id": "etablissement", "libelle": "Établissement", "type": "etablissement", "requis": true},
    {"id": "poste", "libelle": "Poste", "type": "choix", "requis": true,
     "options": ["Equipier Polyvalent", "Manager"], "placeholder": "Poste"},
    {"id": "nationalite_etrangere", "libelle": "Nationalité étrangère", "type": "texte",
     "requis_si": ["nationalite", "Autres"], "placeholder": "NationaliteEtrangere"},
    {"id": "cni", "libelle": "Carte Nationale d'Identité (recto-verso)",
     "type": "piece_jointe", "role": "identite", "requis": true, "max_fichiers": 2}
  ],
  "roles": {"nom": "nom_prenom", "email": "email", "date_debut": "date_debut"}
}
```

Types de champs : `etablissement`, `choix`, `texte`, `texte_long`, `date`, `email`, `piece_jointe`. `placeholder` relie un champ à un jeton des modèles. `requis_si: [champ, valeur]` rend un champ obligatoire, et visible, seulement quand un autre champ vaut cette valeur. `version` est recopié dans chaque dossier (`formulaire_version`) pour lire un vieux dossier après un changement de formulaire.

Tout champ `piece_jointe` doit porter un `role` — c'est lui, et non l'`id`, qui nomme le fichier sur disque (`identite.pdf`, puis « Identité - NOM Prénom.pdf » une fois le dossier ouvert) et qui permet de reconnaître une pièce déjà déposée. Un champ `piece_jointe` sans `role` n'est pas rattrapé par le garde-fou : il lève un `KeyError` au premier affichage du suivi. `store.LIBELLES_PIECE` donne un libellé humain aux rôles connus (`identite`, `carte-vitale`, `rib`, `contrat`, `contrat-signe`, `accuse-dpae`, `fiche-salarie`) ; un rôle absent de cette table sert de libellé tel quel. `max_fichiers` (défaut 1) autorise plusieurs fichiers pour un même rôle : le premier garde le nom nu, les suivants prennent un suffixe `_2`, `_3`…

Ce `role`-là nomme une **pièce jointe** ; à ne pas confondre avec les **rôles du formulaire** ci-dessous, qui désignent les champs que le moteur sait lire.

Rôles du formulaire

Le moteur ne lit **aucun identifiant de champ en dur** ; il passe par un vocabulaire fermé de rôles déclarés dans `roles` :

Rôle	Obligatoire	Usage
`etablissement`	oui	Mentions légales, cabinet, couloir de contrat, arborescence
`poste`	oui	Modèle de contrat, grille de salaire, arborescence
`nom`	oui	Nom du répertoire, noms de fichiers, mails
`email`	oui	Email du candidat ; jamais destinataire d'un mail (sans `manager_email`, le lien de correction part à `mails.rh`)
`prenom`, `nom_usage`	non	Identité quand nom et prénom sont saisis séparément
`date_debut`	non	Colonne « Début », alerte de retard, alerte CDD

Un rôle non déclaré retombe sur son identifiant historique (`etablissement`, `poste`, `nom_naissance`, `email_demandeur`, `date_debut`…) : une configuration sans `roles` garde le comportement d'origine.

`societes.json`

```
[
  {
    "nom": "Basilic Café",
    "siren": "789 456 123",
    "comptable_email": "compta@cabinet.fr",
    "mentions": {"RaisonSociale": "BASILIC CAFE SAS", "RCS": "RCS Paris 789 456 123",
                 "SiegeSocial": "…", "ConventionCollective": "…"},
    "etablissements": [
      {"nom": "Paris Bastille", "siret": "789 456 123 00014", "contrat": "genere",
       "manager_email": "bastille@basiliccafe.fr", "groupe": "RESTAURANTS",
       "mentions": {"AdresseEtablissement": "…"}},
      {"nom": "Lyon Confluence", "siret": "789 456 123 00022", "contrat": "depose"}
    ]
  }
]
```

`contrat` vaut `genere` ou `depose` et fixe le couloir. `manager_email` reçoit les liens de correction. `comptable_email` reçoit le récapitulatif hebdomadaire de la société (repli : `mails.comptable_defaut`).

Sociétés et établissements dans Supabase

Optionnel. Quand `instance.json` contient un bloc `supabase`, **la base est la source des sociétés et des établissements** ; `societes.json` n'est plus qu'une copie de secours.

```
"supabase": {"url": "https://<projet>.supabase.co", "cle": "<clé secrète du projet>"}
```

Sans ce bloc (ou avec `url` ou `cle` vide), rien ne change : `societes.json` est lu et écrit comme avant. `config.supabase_actif()` dit lequel des deux modes est en cours.

> **La clé donne un accès complet à la base** (clé « secret » / `service_role`, qui contourne les règles RLS). Elle vit dans `instance.json` comme le secret HMAC : jamais dans Git, jamais dans une capture. Une clé « publishable » ne suffit pas tant que la lecture n'est pas autorisée par une policy ; `config.py` n'utilise que l'API REST (PostgREST), sans bibliothèque Supabase.

Schéma attendu

Le dépôt ne contient pas de script de migration. Le schéma ci-dessous a été **vérifié sur une base en service** : colonnes, types, valeurs par défaut et clé étrangère `societe_id → societes.id` sont conformes. Les contraintes `unique` n'ont pas été vérifiées, car l'API ne les expose pas : voir *Database → Tables* dans Supabase. Deux tables, une société a plusieurs établissements :

```
create table societes (
  id              bigint generated always as identity primary key,
  nom             text not null unique,
  siren           text not null default '',
  comptable_email text not null default '',
  mentions        jsonb not null default '{}',   -- RaisonSociale, FormeCapital, GreffeRCS, RCS…
  ordre           int  not null default 0
);

create table etablissements (
  id            bigint generated always as identity primary key,
  societe_id    bigint not null references societes(id),
  nom           text not null,
  siret         text not null default '',
  groupe        text not null default '',       -- libre (suggestions : groupes déjà utilisés)
  manager_email text not null default '',
  contrat       text not null default '',       -- '' ou 'genere' = généré ; 'depose' = déposé
  mentions      jsonb not null default '{}',    -- AdresseEtablissement…
  ordre         int  not null default 0,
  unique (societe_id, nom)
);
```

Les contraintes `unique` sont recommandées, pas imposées par le code : l'écran vérifie déjà les doublons avant d'écrire, et un conflit (HTTP 409) de la base est rendu par « Ce nom est déjà utilisé ». Les colonnes sont celles du `select` de `_societes_distantes()` ; en ajouter une ne casse rien, en renommer une casse la lecture.

Lecture, cache et copie de secours

Événement	Comportement
Lecture réussie	Résultat mis en cache **2 minutes** (`_SUPABASE_TTL`), `societes.json` réécrit (écriture atomique, seulement si le contenu change)
Supabase en panne ou réponse invalide	Repli sur `societes.json` , **sans retenter pendant 30 s** (`_SUPABASE_REPRISE`) : une panne ne ralentit pas chaque page. Le journal du serveur dit pourquoi (`[supabase] lecture impossible…`)
Réponse vide ou tronquée (aucune société, société sans nom, aucun établissement)	Refusée comme une panne : **la copie locale n'est jamais écrasée par une liste vide**
Démarrage	La première lecture se fait au premier besoin ; sans réseau, l'instance démarre sur la copie, à condition que `societes.json` existe et soit valide

Conséquences à retenir :

- **Ne plus éditer `societes.json` à la main** quand Supabase est actif : il est écrasé à la prochaine lecture réussie. On modifie la base (onglet **Établissements**, ou le *Table Editor* de Supabase) et on attend au plus 2 minutes.
- C'est la seule exception à « l'app n'écrit jamais dans `config/` » avec le bloc `utilisateurs` (via `/compte`).

- Le bilan de `configurer.py` affiche un avertissement quand Supabase est actif.

Mettre à jour la base

Besoin	Comment
Ajouter ou modifier un établissement, changer un mail	Onglet **Établissements** (écrit dans Supabase, relit aussitôt)
Corriger un lot de lignes, supprimer un établissement	*Table Editor* ou SQL dans Supabase ; visible au plus 2 minutes plus tard
Première mise en service à partir d'un `societes.json` existant	Créer les tables, importer les sociétés puis les établissements (`societe_id` de la société parente, `ordre` = position dans le fichier), puis ajouter le bloc `supabase` : la première lecture réécrit `societes.json` à l'identique

Avant de supprimer ou renommer un établissement, vérifier qu'aucun dossier en cours ni aucune règle `templates` ne le cite : sa clé est « Société / Établissement ». Les dossiers dont l'établissement a disparu restent consultables (le repli est prévu pour valider, rejeter et lister), mais ne peuvent plus générer de contrat.

`grille.json`

Un objet : `champ_heures` à la racine (l'id du champ du formulaire qui porte la durée hebdomadaire), et une liste `postes` où chaque ligne prend l'une des deux formes — forfait mensuel, ou barème par durée :

```
{
  "champ_heures": "temps_travail",
  "postes": [
    {"poste": "Manager", "mensuel": 2800},
    {"poste": "Equipier",
     "bareme": {"24": {"chiffres": "1 480,08"},
                "35": {"chiffres": "1 801,80"}}}
  ]
}
```

Le poste du dossier est rapproché d'une ligne par mots normalisés, la plus spécifique gagnant : « Equipier Polyvalent » prend la ligne « Equipier », « Assistant Manager » ne prend pas celle de « Manager ». `mensuel` l'emporte sur `bareme` s'ils cohabitent ; le montant du barème (`chiffres`) est servi tel quel, sans recalcul, pour qu'aucun arrondi ne diverge du contrat papier. `{{SalaireLettres}}` n'est **pas** une seconde clé à saisir : le moteur la dérive de `chiffres` (`contrat.lettres_fr`), une seule source pour les deux jetons. Une clé `lettres` encore présente dans un `grille.json` ancien est tolérée et ignorée.

Le garde-fou refuse un barème qui ne couvre pas toutes les options du champ d'heures, et refuse deux postes du formulaire qui retomberaient sur la même ligne (le second sortirait au tarif du premier, sans rien signaler). Il refuse aussi un `chiffres` qui n'est pas un nombre (prose, case vide d'espaces, `1.2.3`) : la lecture tombait alors à zéro sans erreur et le contrat sortait signé avec « zéro » en toutes lettres à côté du bon montant en chiffres. Les écritures acceptées sont `1 480,08` , `1.480,08` et `1480.08` .

---

7. Moteur de contrats

Principe

`contrat.remplir()` est le point d'entrée : il calcule les valeurs de tous les jetons puis remplace chaque `{{Jeton}}` du modèle. Un modèle `.docx` est traité paragraphe par paragraphe avec python-docx (les runs sont préservés) ; un modèle `.html` est rempli en texte puis converti en `.docx` par htmldocx. **Le contrat généré est toujours un `.docx`.**

Sources des valeurs

Source	Exemples
Champs du formulaire (`placeholder` du champ)	`{{NomPrenom}}`, `{{DateDebut}}`, `{{TempsTravail}}`
Mentions de la société et de l'établissement	`{{RaisonSociale}}`, `{{Siret}}`, `{{AdresseEtablissement}}`
Grille de salaire	`{{SalaireChiffres}}`, `{{SalaireLettres}}` (dérivé de `SalaireChiffres`, jamais saisi à part)
Saisie RH (`saisie_rh`)	`{{NumeroCarteBTP}}`
Règles dérivées (`derives`)	`{{Nationalite}}`, `{{DureeMensuelle}}`, `{{BlocAutorisationTravail}}`
Calculées par le moteur	`{{Aujourdhui}}`, dates en toutes lettres
Pièces	`{{PiecesFournies}}`, `{{PiecesManquantes}}`

Règles dérivées

Deux formes, évaluées dans l'ordre du fichier (une règle peut utiliser un jeton calculé par une règle précédente) :

```
{"placeholder": "DureeMensuelle", "format": "mensualise", "de": "TempsTravail"}
```

```
{"placeholder": "BlocAutorisationTravail",
 "si": ["nationalite", "==", "Autres"],
 "alors": "<p>Titre « {{TypeAutorisation}} » valable jusqu'au {{DateFinValidite}}.</p>",
 "sinon": ""}
```

Formats disponibles : `lettres` (nombre en toutes lettres, centimes compris ; **jamais** « euro(s) » ni « centime(s) » — c'est la prose du modèle qui écrit l'unité, ex. « {{SalaireChiffres}} euros bruts ({{SalaireLettres}}) »), `mensualise` (heures hebdomadaires × 52 / 12), `nombre` (« 24H » → « 24 »), `majuscules`, `date_longue`.

Vérification

- `config.verifier()` refuse au démarrage tout jeton d'un modèle actif sans source (contrats **et** fiche salarié), ainsi qu'une règle `templates` mal formée ou pointant un champ/une option/une clé d'établissement inconnus (`_verifier_regles`).
- `python doctor.py` génère réellement un contrat pour chaque croisement (poste × temps partiel × établissement × les champs dont dépendent le modèle ou le salaire) **et la fiche salarié**, et signale les cas sans modèle, exclus volontairement (`modele: null`), ou avec une valeur vide.
- `python placeholders.py` écrit `PLACEHOLDERS.md` , la liste exacte des jetons valides pour cette instance, à remettre au client qui balise ses modèles.

---

8. Mails

Modèles

Un fichier `.txt` par mail dans `config/mails/` : la première ligne `Objet: …` , puis le corps. Les `{jetons}` sont ceux passés par le code à `mails.envoyer()` .

Modèle	Déclencheur	Destinataire
`nouvelle_soumission`	Envoi ou correction du formulaire	`mails.rh`
`rejet`	Rejet par la RH	`manager_email` de l'établissement, repli : email du formulaire
`rappel_dpae`	Acceptation de la demande	`mails.dpae` , repli : `mails.rh`
`recap_hebdo`	`recap.py` (cron)	`comptable_email` de chaque société, copie `mails.recap` ou `mails.rh`

Transport

Mode	Comportement
`console`	Écrit un `.eml` dans `data/mails/` et l'affiche sur la sortie standard
`smtp`	Envoi réel : STARTTLS avec vérification du certificat, puis authentification si `utilisateur` est renseigné

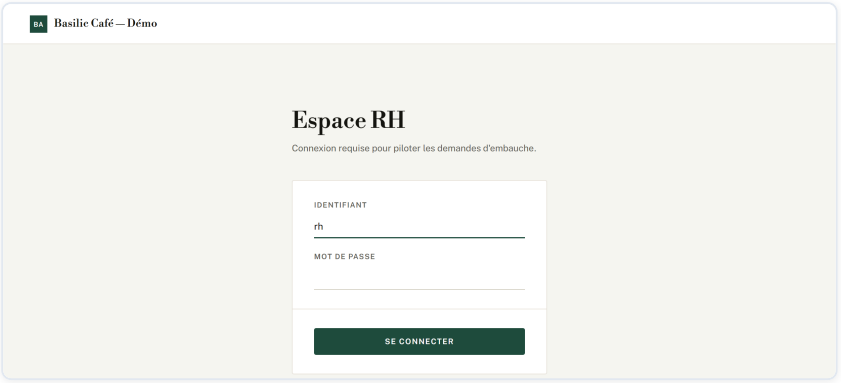
La variable d'environnement `MAILS_MODE` surclasse `mails.mode` . Un échec d'envoi n'interrompt jamais l'action métier : il est journalisé (`type: mail_echoue`) et signalé à l'écran.

---

9. Sécurité

Authentification et session

Comptes dans `instance.json` → `utilisateurs`, mots de passe hachés par `werkzeug.security` (scrypt). Session Flask signée par le secret de l'instance, cookie `HttpOnly`, `SameSite=Lax`, `Secure` hors `DEBUG`, durée 12 heures.



L'écran de connexion

Rate-limit et honeypot

Point d'entrée	Plafond	Fenêtre glissante
Formulaire public et correction (`POST /` , `POST /corriger/…`)	10 envois par adresse IP	5 minutes
Connexion (`POST /login`)	10 tentatives par adresse IP	15 minutes

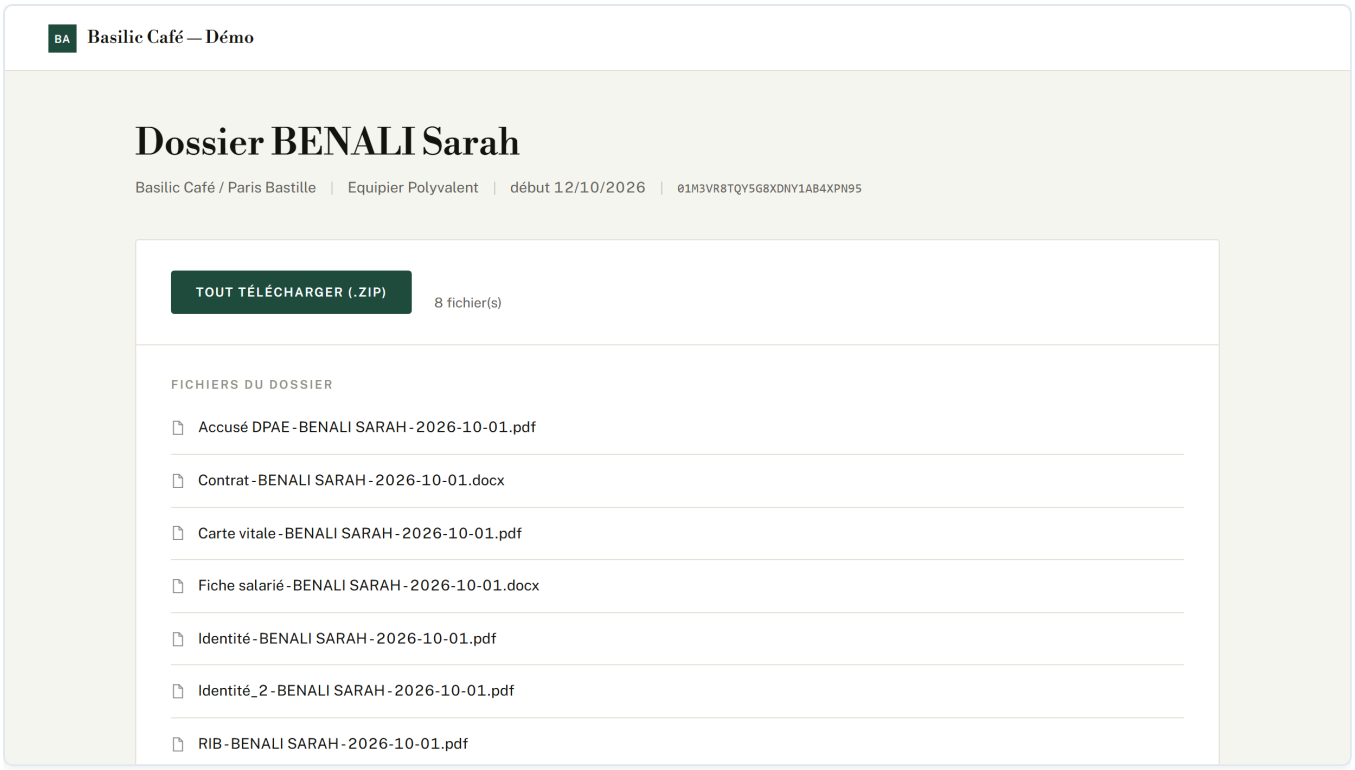
Le champ caché `website` du formulaire est un honeypot : rempli, la demande est acceptée en apparence mais ignorée. L'adresse IP vient de `X-Forwarded-For` uniquement si `PROXIES` déclare le nombre réel de reverse proxies ; sinon l'en-tête est ignoré.

Liens signés

Les liens de correction et les liens du cabinet sont signés HMAC-SHA256 avec le secret de l'instance :

```
<usage>.<ULID>.<horodatage d'émission>.<epoch>.<signature tronquée à 32 hex>
```

`store.verifier_lien()` contrôle la signature, l'usage, l'âge (30 jours) et l'`epoch` du dossier. Incrémenter l'epoch révoque tous les liens émis : `link_epoch` à chaque rejet, `lien_comptable_epoch` au renvoi, à l'abandon et à la purge. Chaque accès à un lot est journalisé avec l'adresse IP.



La page du lot, accessible par le seul lien signé

Téléversements

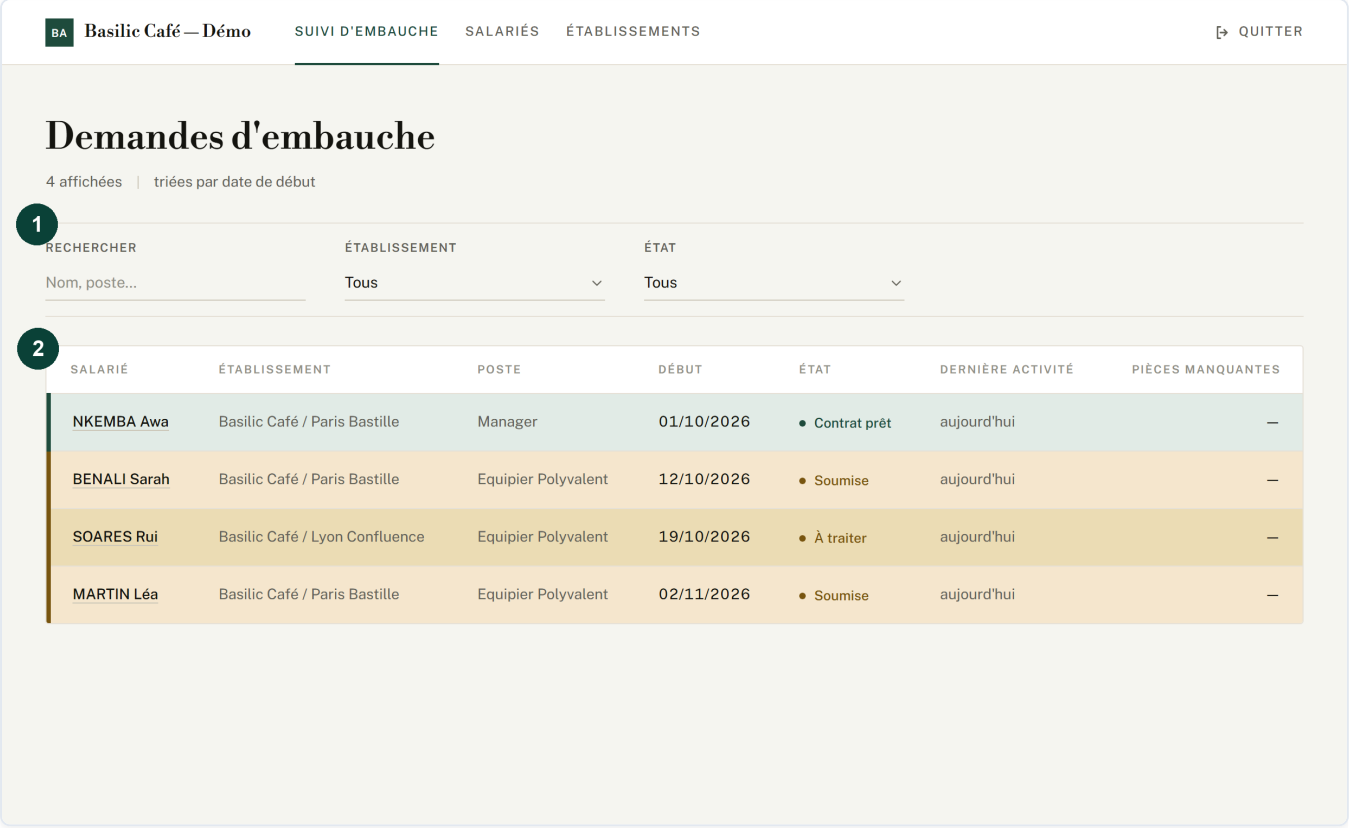
Liste blanche d'extensions (`.pdf` , `.jpg` , `.jpeg` , `.png` ; `.docx` en plus pour les contrats), **15 Mo par fichier** (`store.TAILLE_MAX` , vérifié après écriture d'un temporaire) et 40 Mo par requête (`MAX_CONTENT_LENGTH`), noms de fichiers reconstruits par l'application (jamais le nom envoyé). Les fichiers sont servis avec `X-Content-Type-Options: nosniff` .

10. Routes

Publiques

Méthode	Route	Rôle
GET, POST	`/`	Formulaire de demande (POST classique ou `fetch` avec en-tête `X-Requested-With`)
GET, POST	`/corriger/<jeton>`	Correction d'une demande rejetée
GET	`/lot/<jeton>`	Page du dossier pour le cabinet
GET	`/lot/<jeton>/zip`	Archive du dossier
GET	`/lot/<jeton>/fichier/<bucket>/<nom>`	Un fichier du dossier
GET, POST	`/login` · GET `/logout`	Session RH

Espace RH (session requise)



Le suivi, point d'entrée de l'espace RH

Méthode	Route	Rôle
GET	`/suivi`	Dossiers en cours (tout sauf RemisComptable), filtres établissement et état
GET	`/salaries`	Dossiers remis au comptable, groupés par établissement
GET	`/etablissements`	Liste des établissements : adresse, mail, SIRET
GET, POST	`/etablissements/nouveau`	Créer un établissement, dans une société existante ou une société neuve
GET, POST	`/etablissements/modifier?cle=Société / Établissement`	Modifier un établissement et, avec lui, la fiche de sa société
GET, POST	`/etablissements/recuperer?cle=…`	Importer les salariés d'un établissement depuis le SharePoint (404 sans bloc `sharepoint`)
GET, POST	`/etablissements/completer?cle=…`	Tableau des champs à compléter des dossiers importés ; l'enregistrement est le POST
POST	`/etablissements/completer/excel`	Lire des exports Excel et **proposer** leurs valeurs (rien n'est écrit)
POST	`/sharepoint/connexion`, `/sharepoint/deconnexion`	Ouvrir ou fermer la session Microsoft de la RH
GET	`/dossier/<uid>`	Fiche d'un dossier
GET	`/dossier/<uid>/fichier/<bucket>/<nom>`	Téléchargement d'un fichier
POST	`/dossier/<uid>/valider`	Accepter : ouvre le dossier, fiche salarié, rappel DPAE, contrat si couloir généré
POST	`/dossier/<uid>/rejeter`	Rejeter : motif, commentaire, mail avec lien de correction
POST	`/dossier/<uid>/renvoyer-correction`	Renvoyer le lien de correction (proposé seulement si le mail de rejet a échoué)
POST	`/dossier/<uid>/contrat`	Générer le contrat (saisie RH, ou reprise après échec)
POST	`/dossier/<uid>/contrat-depose`	Déposer le contrat (couloir déposé)
POST	`/dossier/<uid>/contrat-signe`	Déposer le contrat signé
POST	`/dossier/<uid>/signature-envoyer`, `…/signature-verifier`	Signature électronique (si activée)
POST	`/dossier/<uid>/dpae-faite`	Enregistrer l'accusé DPAE
POST	`/dossier/<uid>/remettre`	Remettre au cabinet : copie dans `COMPTA/`
POST	`/dossier/<uid>/renvoyer`	Refaire la copie et révoquer le lien du cabinet

Méthode	Route	Rôle
POST	`/dossier/<uid>/abandonner`	Abandonner
POST	`/dossier/<uid>/date-entree`	Saisir la date d'entrée d'un dossier importé
POST	`/dossier/<uid>/fiche`	Générer la fiche salarié d'un dossier importé, ses champs complétés
GET, POST	`/purger`	Aperçu puis exécution de la purge

---

11. Installation et déploiement

Nouveau client

```
pip install -r requirements.txt
python installer.py "Basilic Café" /srv/basilic
# stockage dans un dossier synchronisé :
python installer.py "Basilic Café" /srv/basilic "C:/Users/rh/OneDrive - Basilic/Embauches"
```

L'installateur copie `config.exemple/` vers `/srv/basilic/config/`, génère le secret HMAC et un mot de passe RH initial (affiché en fin d'installation, à changer avec `configurer.py compte`), et écrit le bloc `stockage`. Il reste à remplir la configuration :

```
CONFIG_DIR=/srv/basilic/config python configurer.py            # bilan : ce qui manque
CONFIG_DIR=/srv/basilic/config python configurer.py --assister  # assistant interactif
CONFIG_DIR=/srv/basilic/config python placeholders.py          # fiche des jetons pour le client
CONFIG_DIR=/srv/basilic/config python doctor.py                # couverture des modèles
```

Lancement

```
CONFIG_DIR=/srv/basilic/config DONNEES=/srv/basilic/data python app.py   # production (waitress)
DEBUG=1 python app.py                                                     # développement (werkzeug)
```

Variable	Défaut	Rôle
`CONFIG_DIR`	`config/`	Répertoire de configuration de l'instance
`DONNEES`	`data/` à côté de `config/`	Répertoire de données ; **l'emporte sur** `stockage.chemin`, y compris en mode `dossier`
`HOST`, `PORT`	`127.0.0.1`, `5000`	Adresse et port d'écoute
`PROXIES`	`0`	Nombre de reverse proxies devant l'application ; active la lecture de `X-Forwarded-For`
`DEBUG`	absent	Serveur de développement, cookie sans `Secure`
`MAILS_MODE`	absent	Surclasse `mails.mode` (`console` ou `smtp`)

Reverse proxy

L'application écoute en local ; un reverse proxy porte le TLS et transmet l'IP d'origine.

```
# Caddy
embauche.basiliccafe.fr {
    reverse_proxy localhost:5000
}
```

Lancer alors avec `PROXIES=1`. Waitress efface par défaut les en-têtes `X-Forwarded-*` non déclarés : `PROXIES` est lu par `app.py` pour configurer waitress **et** Flask, un réglage seul ne suffit pas.

Tâche hebdomadaire

```
CONFIG_DIR=/srv/basilic/config DONNEES=/srv/basilic/data python recap.py
```

À planifier une fois par semaine (cron, tâche planifiée Windows). Sans cette tâche, les dossiers remis n'arrivent jamais au cabinet.

Chaque cabinet repart de son dernier envoi réussi, noté dans `DONNEES/recap.json` (`--jours` , 7 par défaut, ne règle que le premier envoi). Un envoi raté laisse ses dossiers en attente pour le passage suivant et s'affiche en bandeau sur l'écran « Salariés » ; l'état est réécrit après chaque cabinet. `recap.py` n'écrit jamais un `dossier.json` : il tourne dans un second process, et `store._verrou` n'exclut que les threads d'un même process.

Sauvegarde et mise à jour

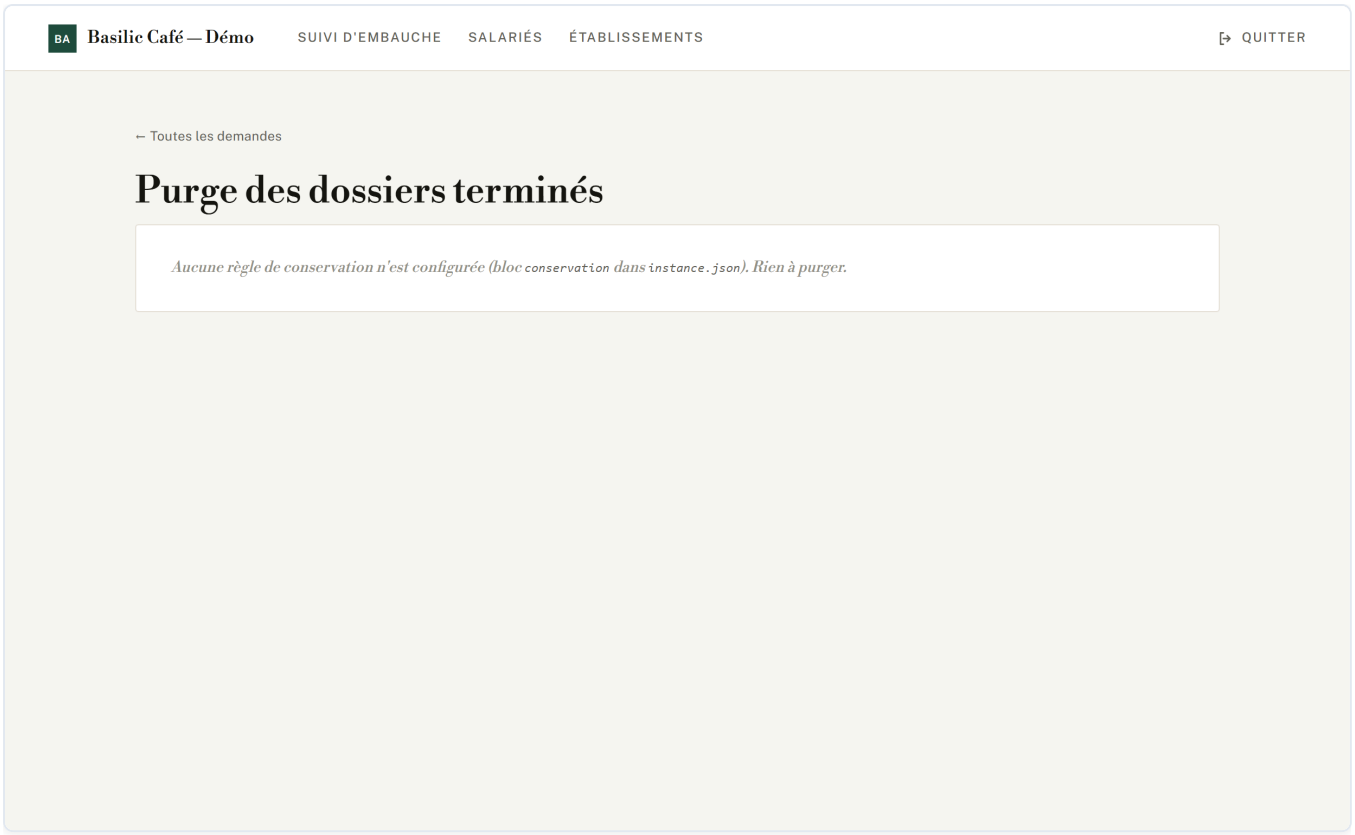
```
tar czf sauvegarde-$(date +%F).tar.gz /srv/basilic/config /srv/basilic/data
```

Restaurer = décompresser et relancer. Mettre à jour = remplacer le code (`git pull`) et relancer : `config/` et `data/` ne sont jamais modifiés par le code, il n'y a rien à migrer.

---

12. Exploitation

Besoin	Commande ou écran
Vérifier la configuration	`python configurer.py` (bilan), garde-fou au démarrage
Tester l'envoi de mail	`python configurer.py mail votre@adresse`
Ajouter un compte RH	`python configurer.py compte identifiant`
Vérifier les modèles de contrat	`python doctor.py`
Appliquer une modification de `config/`	Redémarrer l'application : `config/` est relu au démarrage
Voir ce que la purge effacerait	`python purger.py` (lecture seule) ; `/purger` dans l'application pour exécuter
Suivre ce que fait le serveur	Sortie standard : les deux lignes de démarrage et les erreurs Python. **Waitress ne journalise pas les requêtes** — l'historique métier est le journal de chaque dossier
Mails en mode console	`data/mails/*.eml`



L'aperçu de purge : lecture d'abord, écriture seulement au clic

Le démarrage refuse de servir si un autre serveur tient le verrou du stockage, si la configuration est invalide, ou si le dossier de stockage synchronisé n'est pas accessible en écriture. Le message d'erreur nomme la cause.

---

13. Tests

Aucun framework : chaque fichier est un script exécutable, la suite est verte quand tous sortent avec le code 0.

```
python tests/test_parcours.py         # les deux couloirs de bout en bout, fiche, récap
python tests/test_ecrans.py           # rendu de chaque écran dans chaque état
python tests/test_formulaire.py       # soumission fetch (JS) du formulaire public
python tests/test_pieces_multi.py     # pièces multi-fichiers (recto/verso)
python tests/test_valeurs_contrat.py  # valeurs imprimées dans le contrat (grille, mensualisation)
python tests/test_produit.py          # garde-fou, fiche salarié, version de configuration
python tests/test_clients.py          # deux instances isolées, étanchéité multi-client
python tests/test_etablissements.py   # onglet Établissements : liste, ajout, modification
python tests/test_supabase_societes.py  # lecture Supabase, copie de secours, écriture (transport
factice)
python tests/test_login.py            # rate-limit, session, PROXIES à travers waitress
python tests/test_mails.py            # mode console, surcharge, STARTTLS vérifié
python tests/test_configurer.py       # bilan, assistant, comptes, test SMTP
python tests/test_doctor.py           # couverture de configuration
python tests/test_purge.py            # éligibilité, effacement, journal conservé
python tests/test_signature.py        # Yousign contre un transport factice
python tests/test_import.py           # import SharePoint (transport Graph factice), doublons, reprise
python tests/test_excel.py            # lecture .xlsx, conversion, rapprochement, écran « Compléter »
```

Les tests utilisent `config/` (fixture versionnée, client fictif) ou construisent une configuration temporaire. `test_clients.py` vérifie que tout nouveau module racine est bien couvert : y ajouter tout nouveau `.py` à la racine.

---

14. Étendre une instance

Ajouter un champ au formulaire

1. Ajouter l'entrée dans `formulaire.json` → `champs` (identifiant, libellé, type, `requis` ou `requis_si`).
2. S'il alimente un jeton : `"placeholder": "MonJeton"`, puis `{{MonJeton}}` dans les modèles.
3. S'il joue un rôle pour le moteur (nom, email, date de début) : le déclarer dans `roles`.
4. `python placeholders.py` puis `python doctor.py`, et redémarrer l'application.

Ajouter un modèle de contrat

1. Déposer le `.docx` ou `.html` dans `config/contrats/`.
2. L'associer dans `instance.json` → `templates`, en forme liste si le choix dépend d'autres champs : `json {"quand": {"poste": "Equipier Polyvalent", "type_contrat": "CDD"}, "modele": "Equipier_CDD.html"}`
3. `python doctor.py` pour vérifier que chaque croisement a un modèle et aucune valeur vide.

Ajouter un établissement, une société, un compte

- Établissement ou société : onglet **Établissements** de l'espace RH (`/etablissements/nouveau`), sans redémarrage. Les champs obligatoires sont contrôlés côté serveur (`_saisie_etablissement`). Sans bloc

`supabase`, l'écran écrit `societes.json` (`config.sauver_etablissement`, écriture atomique, cache mis à jour) ; avec, il écrit dans Supabase puis relit.

- À la main : `societes.json` → `etablissements` (nom, SIRET, `contrat`, `manager_email`), puis redémarrer — uniquement sans Supabase.
- Compte : `python configurer.py compte identifiant`.

Ajouter un mail

1. Créer `config/mails/<nom>.txt` : première ligne `Objet: …`, puis le corps avec des `{jetons}`.
2. Appeler `mails.envoyer("<nom>", destinataires, **valeurs)` depuis le code.

Personnaliser les motifs de rejet

`instance.json` → `motifs_ko` : la liste s'affiche telle quelle dans le menu du rejet.

Contrat_Gen — édition du 16 septembre 2026

---

15. Dossiers importés : SharePoint et Excel

Un salarié embauché avant l'outil a déjà un dossier en ordre ailleurs. Deux boutons de l'onglet **Établissements** le font entrer dans l'application sans repasser par le parcours : **Récupérer les dossiers** (SharePoint) puis **Compléter les dossiers** (tableau et exports Excel). Tout est inactif sans bloc `sharepoint` dans `instance.json`.

Le bloc `sharepoint`

```
"sharepoint": {
  "site": "https://acme.sharepoint.com/sites/rh",
  "racine": "DOSSIERS SALARIES",
  "ignorer": ["Anciens", "Archives"],
  "mots_ignores": ["restaurant"],
  "postes": {"Chefs d'équipe": "Manager", "Direction": ""},
  "dossiers": {"ACME Sud / Marseille Prado": "RESTAURANTS/Marseille"}
}
```

Clé	Rôle
`site`	Adresse du site (`https://<tenant>.sharepoint.com/…`) — obligatoire
`racine`	Dossier qui contient les groupes ; arborescence lue : `<racine>/<groupe>/<établissement>/<poste>/<salarié>/` — obligatoire
`client_id`	Facultatif : application Azure du client. Absent = le client public Office, sans consentement administrateur
`ignorer`	Sous-dossiers sautés à tout niveau (salariés partis, copies)
`mots_ignores`	Mots trop courants pour rapprocher un établissement d'un dossier
`postes`	Dossier de poste SharePoint → poste du formulaire (`""` : poste laissé vide)
`dossiers`	« Société / Établissement » → `GROUPE/Dossier`, quand le nom ne suffit pas

`config._verifier_sharepoint()` en refuse la forme au démarrage. `configurer.py` avertit si `msal` n'est pas installé, ou si le bloc coexiste avec `stockage.mode = "dossier"` (voir plus bas).

Installation : `pip install -r requirements.txt -r requirements-sharepoint.txt`. Sans le bloc, l'application n'importe jamais `msal` ni `requests`.

Récupérer les dossiers (`sharepoint.py`)

1. **Connexion Microsoft** : *device flow* MSAL. La RH saisit un code sur le site de Microsoft avec son propre compte ; l'application ne voit jamais de mot de passe. Le jeton est mis en cache dans `DONNEES/sharepoint-jeton.json`, effacé par **Se déconnecter**.
2. **Dossier de l'établissement** : jamais choisi par la RH. Dans l'ordre : le dossier déjà lié par un import précédent, celui écrit dans `sharepoint.dossiers`, sinon le seul dossier nettement le plus ressemblant (`_ressemblance`). Ambigu ou introuvable : rien n'est proposé, l'installateur renseigne `dossiers`. Un dossier lié à un autre établissement n'est jamais rendu, et le dossier posté est revérifié côté serveur (404 sinon).
3. **Liste à cocher** : un salarié par dossier. Déjà importé : non cochable. Même nom qu'un salarié du parcours normal : **signalé et décoché par défaut, jamais bloqué** — un vrai homonyme reste importable.
4. **Import en tâche de fond**, un seul à la fois pour toute l'application (deux établissements pointant le même dossier passeraient sinon tous deux le test d'idempotence). L'écran affiche l'avancement.

Pris dans chaque dossier salarié : les fichiers à sa racine et ceux des sous-dossiers `CONTRAT` et `FICHE PERSONNELLE` (tolérant à la graphie). **Rien d'autre** (dossiers médicaux, disciplinaires). Un fichier de plus de 50 Mo est laissé à la RH et signalé ; l'import s'arrête avant de laisser moins de 1 Go de disque libre.

`store.importer()` crée le dossier directement à l'état `RemisComptable` : tout s'écrit dans `DONNEES/.import/<uid>` puis est déplacé d'un bloc, donc un échec ne laisse aucun dossier à moitié né ; `store.vider_imports()` nettoie les chantiers d'un arrêt brutal. La clé de source (`import_cle` = `sharepoint:<id du dossier>`) rend l'import rejouable sans doublon. Les fichiers gardent leur nom d'origine, raccourci pour que le chemin final tienne sous 240 caractères. Seuls quatre champs sont remplis (`sharepoint.champs_de()`) : établissement, nom, poste (dossier de poste, via `postes` ou une option du

formulaire au pluriel près) et temps de travail, lu dans le suffixe du dossier salarié (« DUPONT Marie (18h) » → `18H`, `analyser_nom()`) s'il figure dans les options du champ.

Conséquences sur un dossier importé (`import_cle` présent) :

Aspect	Comportement
Pièces manquantes, « signé manquant »	Jamais affichés : les rôles des fichiers sont inconnus, le dossier était en ordre à la source
Lien et mail du cabinet	Masqués et refusés : le cabinet le connaît déjà, un clic l'enverrait en double au mail hebdomadaire
Récapitulatif hebdomadaire	Absent : l'entrée `import` a `de == vers`, `est_remise()` ne la compte pas
Purge	Jamais tant qu'il est en poste : seulement une fois `Parti` (si `Parti` figure dans `conservation.apres`)
Fichiers hors liste blanche (`.html`, `.svg`…)	Toujours servis en pièce jointe, jamais affichés dans le navigateur

Compléter les dossiers (`excel.py`)

Le SharePoint ne donne ni état civil, ni contrat, ni date d'entrée : `/etablissements/completer` affiche un tableau des champs **requis** du formulaire (y compris sous condition) pour les dossiers importés, en poste, de l'établissement. Le nom nomme le dossier et sert de clé de rapprochement : il ne se modifie pas.

- **Saisie directe** : une cellule vidée efface. Une cellule invalide (date non `jj/mm/aaaa`, valeur hors liste) bloque tout l'enregistrement (422) : rien n'est écrit. Un **verrou optimiste** par ligne (`v:<uid>`, empreinte des champs) refuse d'écraser une ligne enregistrée ailleurs depuis l'affichage.
- **Excel** : la RH envoie un ou plusieurs exports `.xlsx` (logiciel RH, planning). Rien n'est écrit ni conservé : les valeurs sont **proposées** dans les cellules vides, la RH vérifie puis enregistre. Une cellule déjà remplie qui diffère affiche « Excel : … » sans être touchée.
- **Lecture** : stdlib seule (un `.xlsx` est un zip de XML), 1re feuille ; chaque ligne contenant « Nom » et « Prénom » ouvre un en-tête valable jusqu'au suivant ; 20 Mo maximum par membre du zip. Les colonnes reconnues (`excel.convertir`) sont converties vers les champs que **le formulaire du client possède** ; une valeur non reconnue est un *rejet*, affiché et jamais écrit. Plusieurs valeurs différentes pour une même personne forment un *conflit*, jamais rempli.
- **Rapprochement** par `sharepoint.cle_nom` (mots du nom sans ordre, accents ni casse). Zéro ou plusieurs dossiers en poste à ce nom : la ligne est « non rattachée » et la RH choisit le dossier à la main. Un nom sans dossier en poste mais qui correspond à un salarié `Parti` est ignoré, plutôt que proposé au rattachement d'un autre dossier.
- **Journal** : `correction` (tableau) ou `completion_excel` (rattachement manuel, champs vides seulement). Il porte les **identifiants** des champs écrits, jamais leurs valeurs. `store.completer()` refuse tout dossier qui n'est pas importé et en poste (sur un `Parti`, la note masquerait la date de sortie).

Sur la fiche d'un dossier importé : **Date d'entrée** (`/dossier/<uid>/date-entree` ; le SharePoint ne la donne pas) et **Générer la fiche salarié** (refusée tant qu'un champ obligatoire est vide ; une fiche venue du SharePoint sous le même nom n'est jamais remplacée).

Sécurité

- Le jeton Microsoft donne accès au SharePoint de la RH. Avec `stockage.mode = "dossier"`, `DONNEES/` est le dossier synchronisé : le jeton partirait avec. D'où l'avertissement de `configurer.py` et la consigne de se déconnecter après chaque import.
- Toutes ces routes exigent la session RH (`@rh`) ; un `cle` d'établissement inconnu, ou un bloc `sharepoint` absent, répond 404.
- Les envois Excel sont limités par `MAX_CONTENT_LENGTH` ; au-delà, un message dans l'écran plutôt que la page brute (413).
- Les heures du journal s'affichent à l'heure locale du serveur (`_horodatage`), pas en UTC.